Civic Security

Avance 1 del MVP: arquitectura y base de datos

Equipo Civic Security

ET201 – Formación de Empresas de Base Tecnológica I
Universidad Católica San Pablo

28 de octubre de 2026

El problema y nuestra propuesta

Problema validado en entrevistas

- La información de seguridad circula en grupos de WhatsApp, pero nadie confía en ella porque no está verificada.
- Muchas personas no reportan por miedo a ser identificadas.

Civic Security

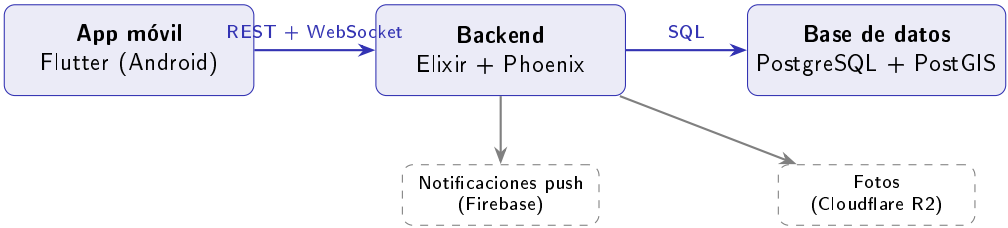
1. Reporte **anónimo** y seguro.
2. **Validación comunitaria**: 3 confirmaciones de cuentas verificadas a menos de 1 km.
3. **Mapa de calor** y alertas por zona en tiempo real.

Qué logramos en este avance

1. **Arquitectura definida y documentada** (v0.2), con las decisiones del equipo incorporadas.
2. **Repositorio público** con historial de commits del equipo.
3. **Base de datos funcionando**: PostgreSQL + PostGIS en Docker.
4. **Backend Phoenix** con verificación de salud (`/health`).
5. **Modelo de incidente** con las reglas de negocio y **10 pruebas automáticas** en verde.

Repositorio: `https://github.com/aaabriceno/Proyecto-MVP-Civic-Security`

Arquitectura: monolito modular



Grano fino (mantenibilidad): el código se divide en módulos por dominio con fronteras claras.

Grano grueso (rendimiento): se despliega como un solo servicio y una sola base de datos, sin la complejidad de microservicios.

Decisiones del equipo

Tema	Decisión
Verificación de persona	DNI, guardando solo un **HMAC** (nunca el número). Una cuenta por DNI.
Zonas de alerta	En el servidor: celda aproximada en claro y **centro exacto cifrado**.
Red P2P	Descartada: sin servidor central no se puede garantizar la validación comunitaria.
Plataforma de la demo	Solo Android.
Fuera del MVP	Entrega cifrada a la PNP, eventos de tráfico, passkeys.

Privacidad por diseño

El anonimato se garantiza por la estructura de los datos, no solo por una política.

Dato	Tratamiento	Por qué
Reporte	Sin campo de usuario ni IP	Nadie puede saber quién reportó
DNI	HMAC con clave secreta	Irreversible; solo detecta duplicados
Centro de zona	Cifrado AES-256-GCM	Se descifra solo en memoria
Fotos	Rostros difuminados, sin EXIF	No exponen a terceros ni la ubicación

Estándares de referencia: OWASP MASVS, OWASP ASVS, NIST SP 800-63B y Ley N.º 29733.

Base de datos: tabla incidentes

```
 Column           | Type
------------------+------------------------
 id               | uuid
 tipo             | character varying(255)
 descripcion      | character varying(500)
 ubicacion        | geography(Point,4326)
 estado           | character varying(255)
 idempotency_key  | uuid
 inserted_at      | timestamp
 updated_at       | timestamp
Indexes:
 incidentes_pkey   PRIMARY KEY (id)
 ..._idempotency_key_index  UNIQUE
 ..._inserted_at_index      btree
 ..._ubicacion_index        gist
```

- **UUID**: los reportes no se pueden enumerar (1, 2, 3...).
- **geography**: distancias reales en metros, para la regla de 1 km.
- **Índice GiST**: búsquedas rápidas por zona del mapa.
- **UNIQUE**: impide reportes duplicados.
- Ninguna columna identifica al usuario.

Reglas de negocio en el código

```
def create_changeset(incident, attrs) do
  incident
  |> cast(attrs, [:tipo, :descripcion, :lat, :lng])
  |> validate_required([:tipo, :lat, :lng])
  |> update_change(:descripcion, &trim_or_nil/1)
  |> validate_length(:descripcion, max: 500)
  |> validate_arequipa()
  |> put_ubicacion()
  |> unique_constraint(:idempotency_key)
end
```

- Solo 5 tipos de incidente válidos.
- Ubicación obligatoria y **dentro de Arequipa**.
- Descripción opcional, máximo 500 caracteres.
- Todo reporte nace como **"Sin verificar"**.
- **Lista blanca** de campos: nadie puede marcar su propio reporte como verificado.
- **Idempotencia**: si la app reintenta por mala señal, no se duplica.

Evidencia de funcionamiento

Base de datos (Docker)

```
$ docker compose ps
civic_db | postgis/postgis:17-3.5
         | Up (healthy) | 127.0.0.1:5432

$ SELECT postgis_version();
3.5 USE_GEOS=1 USE_PROJ=1 USE_STATS=1
```

La base de datos solo acepta conexiones locales (127.0.0.1).

Pruebas automáticas

```
$ mix test --trace test/civic
* crea un incidente válido (sin_verificar)
* rechaza tipo desconocido
* rechaza ubicación fuera de Arequipa
* exige lat y lng
* descripción con solo espacios -> nil
* descripción de 501 caracteres inválida
* misma idempotency_key -> mismo incidente

10 tests, 0 failures
```

Estado del plan del primer entregable funcional

#	Tarea	Estado
1	Base de datos PostgreSQL + PostGIS en Docker	**Hecho**
2	Backend Phoenix con verificación de salud	**Hecho**
3	Modelo de incidente con reglas de negocio	**Hecho**
4	API para crear reportes (`POST /v1/incidents`)	Pendiente
5	API del mapa (`GET /v1/map/snapshot`)	Pendiente
6	Canal en tiempo real	Pendiente
7–10	App Flutter: mapa, reporte y pines en tiempo real	Pendiente

Meta del entregable: un reporte creado en un teléfono Android aparece en el mapa de otro teléfono en tiempo real.